Engine DAQ Telemetry

User Guide

Python build, version 0.1.0

Monitoring only. This application reads data. It cannot stop, load or control the engine or dynamometer. It is not a safety system and does not replace the rig's independent protection, interlocks or emergency stop.

Hardware status. Every driver for real hardware in this version has been tested against software endpoints at most. None has been validated on a physical instrument. Check a new device against a trusted reference before relying on its readings (Section 4).

Contents

1 Getting started **2**
- 1.1 Requirements . . . 2
- 1.2 Starting the application . . . 2
- 1.3 A five-minute tour . . . 2

2 The main window **3**

3 Connecting hardware **4**
- 3.1 Discovering devices . . . 4
- 3.2 The configuration wizard . . . 5
- 3.3 Driver settings . . . 5
- 3.4 Instruments with no driver . . . 5

4 The hardware compatibility matrix **6**
- 4.1 Current interface support . . . 6
- 4.2 Adding your equipment . . . 7

5 Measurement channels **8**
- 5.1 Calibration . . . 10
- 5.2 Derived measurements . . . 10

6 Alarms **11**
- 6.1 The five classifications . . . 11
- 6.2 Alarm lifecycle . . . 11
- 6.3 Reading an alarm . . . 12

7 Validation rules **12**
- 7.1 Cross-channel rules . . . 12

8 Diagnostics **13**

9 Dashboards **13**

10 Engine and compression ratio **14**
10.1 Compression ratio source 14
10.2 Calculations 15

11 Recording and data exchange **15**
11.1 Exporting 15
11.2 Importing 16

12 Experiments and reports **16**
12.1 Reports 17

13 Simulation **17**

14 Saving setups **18**

15 Troubleshooting **18**

16 Known limitations **19**

17 Where your files are **19**

18 Licence **20**

1 Getting started

1.1 Requirements

Python 3.10 or newer, on Windows, Linux or macOS. This version has been tested only on Linux with Python 3.14; Windows and macOS are supported by design but have not been run yet. Nothing else needs installing by hand, and MATLAB is not needed. Some instruments also need their maker's own driver software, such as NI-DAQmx.

1.2 Starting the application

On Windows, double-click `run.bat`. On Linux or macOS, run `./run.sh`. On any system, `python launch_engine_daq.py` does the same.

The first run creates a private environment in `.venv`, installs everything the application needs there, and starts it. That needs internet access, a few minutes and about 1.3 GB of disk. Later runs start straight away and work offline. Nothing is installed outside this folder. If the folder is copied to another computer, the environment rebuilds itself.

All of your data is kept in `workspace/`, so backing up or moving that one folder takes it with you.

The first launch loads a simulated engine rig, so you can try every feature with no hardware.

1.3 A five-minute tour

1. Press **Start monitoring**. The status bar turns green once data is flowing.
2. Open **Simulation**, select *Single-sample spike* and press **Inject selected**. A yellow banner appears.
3. Select *Critical temperature exceeded* and inject it. A red banner appears.
4. Open **Alarms**, select a row to read the diagnosis, and press **Acknowledge**.
5. Open **Diagnostics** to see that health is reported per layer.
6. Press **Stop monitoring**.

2 The main window

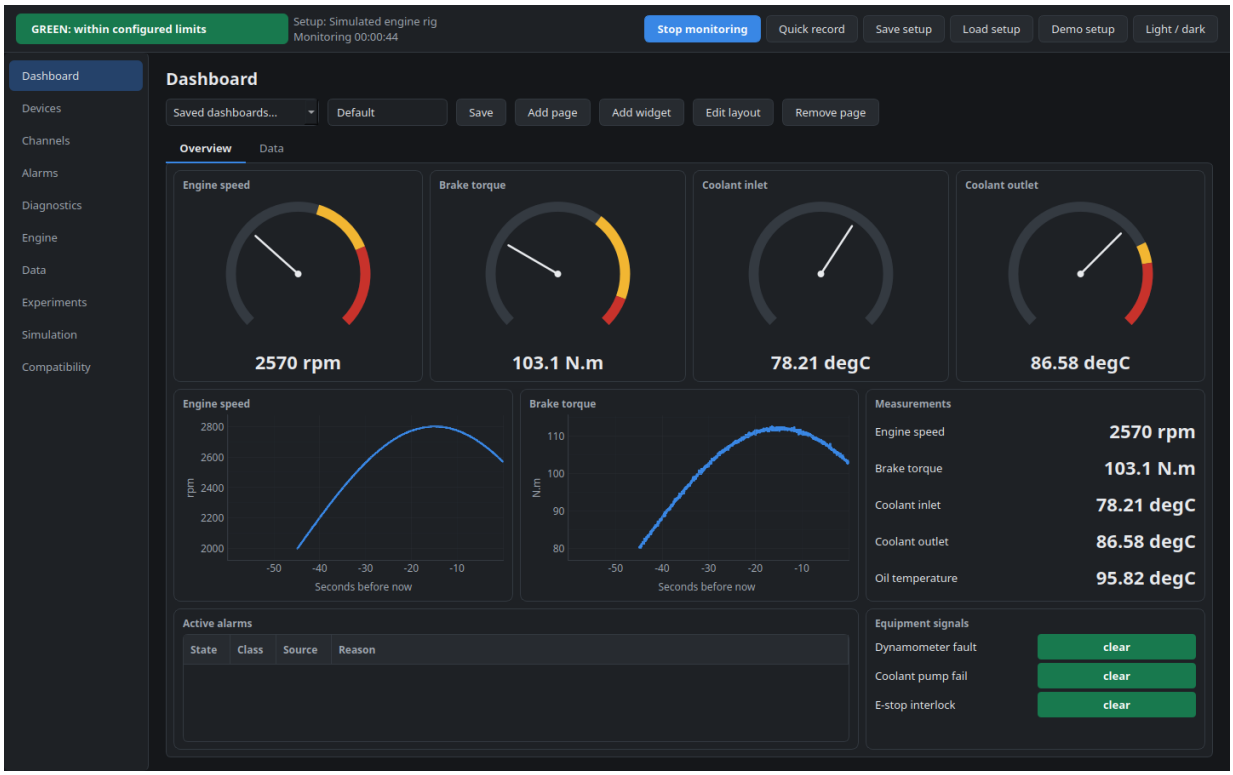


Figure 1: Main window while monitoring a healthy simulated rig.

The top bar shows the overall state, the current setup, how long monitoring has been running and whether a recording is in progress, and holds the monitoring, recording and setup controls. Status columns in the tables use the same colours. Banners for active alarms appear below it and stay there until the alarms are dealt with. Clicking a banner opens the **Alarms** page. Pages are chosen from the list on the left, and **Light / dark** switches the colour theme.

State	Meaning
GREEN	Data is flowing and everything is within configured limits. It does *not* certify that the equipment is healthy; the application only knows what its channels report.
YELLOW	Something is suspicious: an anomalous measurement, a communication problem or a degraded instrument. Nothing has been confirmed about the equipment.
RED	A verified equipment or instrument fault, loss of a critical sensor, or a configured critical limit.
NO DATA	Not monitoring, or no samples are arriving. Idle is never shown as green.

Page	Use it to
Dashboard	Watch live values on layouts you design
Devices	Find hardware, configure it and test connections
Channels	Define what is measured and how it is validated
Alarms	Read, acknowledge and export alarms

Page	Use it to
Diagnostics	See communication, instrument, measurement and equipment health separately
Engine	Enter geometry and manage compression ratio
Data	Export recorded runs and import outside data
Experiments	Define experiments, record runs and build reports
Simulation	Rehearse faults on the simulated rig
Compatibility	See and maintain what hardware is supported

3 Connecting hardware

3.1 Discovering devices

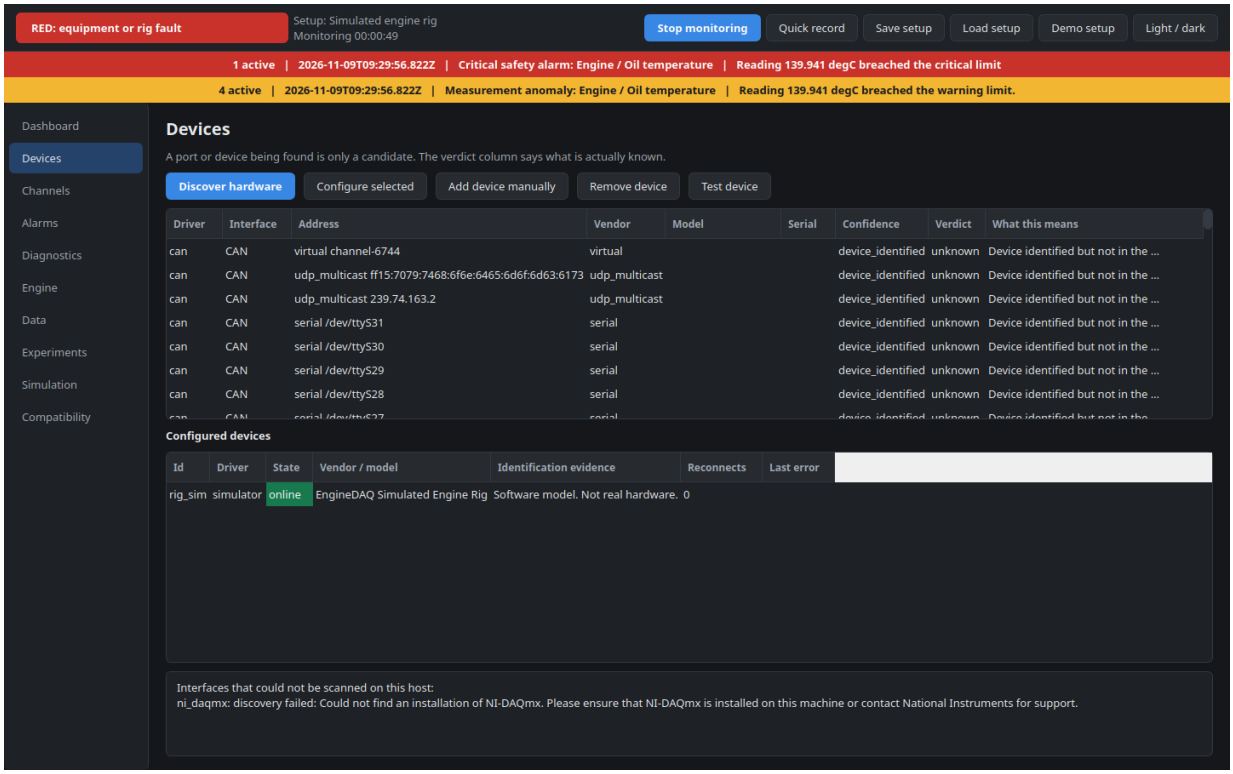


Figure 2: Devices page after a discovery scan.

Devices > Discover hardware lists every candidate it can find. The *Confidence* column says how much is really known:

`port_only` A port exists. What is attached to it, and what protocol it speaks, is unknown.
`device_identified` The interface or device named itself. That does not prove its data can be read correctly.
`protocol_verified` Valid data was received.

The *Verdict* column is the compatibility matrix's position on that device. **`unknown`** means it is not in the matrix and has not been verified. The box at the bottom lists interfaces that could not be scanned on this computer, and why.

3.2 The configuration wizard

Select a candidate and press **Configure selected**, or press **Add device manually**.

1. Choose a driver. Drivers that cannot run on this computer are marked *unavailable*.
2. Enter a device id and fill in the settings. The form is built from what the driver needs.
3. Fill in the channel table: which registers, fields or signals to read.
4. Press **Test connection** and read each step of the result.
5. Press **Save profile** to reuse this configuration, then **Add to setup**.

A passing connection test shows that the link works and frames decode. It does not show that the measurements are accurate.

Several devices can be acquired from at once, and a fault on one does not interrupt the others. A lost device is retried automatically with increasing delays, from 1 s up to 30 s. A device that sends nothing for 10 s is treated as lost.

3.3 Driver settings

Driver	Channel table
`stream`	`hw`: your name for the channel. `field`: column number (*delimited*), key (*keyvalue*) or JSON name (*json*). Set the transport, terminator and frame format above the table. For polled instruments, enter a poll command.
`modbus`	`hw`, `table` (`holdingregs`, `inputregs`, `coils`, `inputs`), `address`, `type` (`uint16`, `int16`, `uint32`, `int32`, `single`, `double`), `serverId`. Check the address offset against the device manual on first use.
`can`	`hw`, `messageId`, `extended`, `startBit`, `length`, `byteOrder` (`intel` or `motorola`), `signed`, `scale`, `offset`. Set the python-can interface and channel above the table. Keep *Listen only* on unless the bus needs acknowledgement from this node.
`ni_daqmx`	`hw`, `device`, `channel`, `measurement`, `rangeLow`, `rangeHigh`, `thermocoupleType`. Needs NI's driver software as well.
`visa`	`hw`, `query`. Every query must end in `?`. Leave the error query enabled: it is what gives real evidence of instrument health.
`opcua`	`hw`, `namespace`, `identifier`.

3.4 Instruments with no driver

If the instrument has a vendor SDK, either write a small bridge in any language that prints one JSON object per line over TCP or UDP and read it with the `stream` driver, or add a driver plugin as described in `docs/DRIVER_GUIDE.md`. Plugins need no change to the application.

4 The hardware compatibility matrix

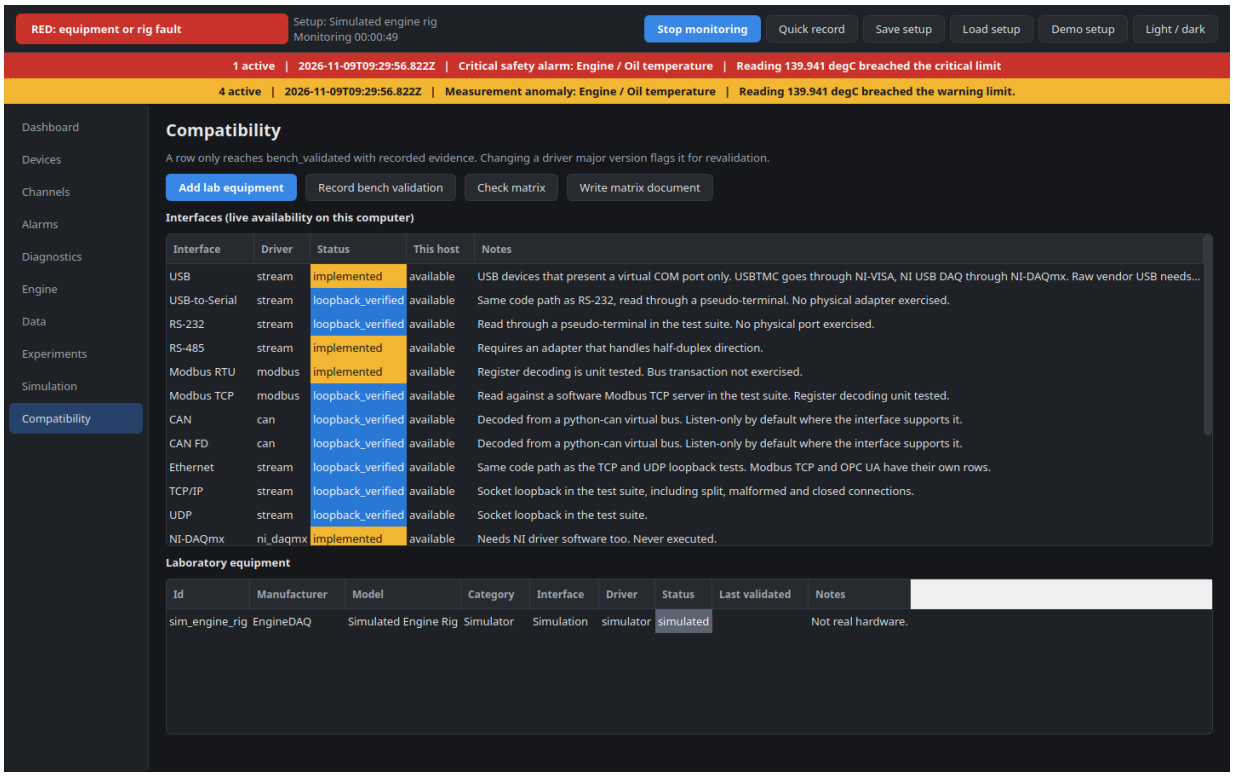


Figure 3: Compatibility page, with live availability on this computer.

The matrix records what hardware works and exactly how far each claim has been checked.

Status	Meaning
`unsupported`	Known not to work, or deliberately excluded.
`planned`	Intended; no driver yet.
`implemented`	Driver written but never run against any endpoint.
`simulated`	Software model only.
`loopback_verified`	Ran end to end against a software endpoint. Not tested on a physical instrument.
`bench_validated`	Verified on a physical instrument, with recorded evidence.
`deprecated`	Still present, scheduled for removal.

4.1 Current interface support

Interface	Driver	Status	Platforms	Notes
USB	`stream`	`implemented`	any	USB devices that present a virtual COM port only. USBTMC goes through NI-VISA, NI USB DAQ through NI-DAQmx. Raw vendor USB needs a plugin or adapter.
USB-to-Serial	`stream`	`loopback_verified`	any	Same code path as RS-232, read through a pseudo-terminal. No physical adapter exercised.
RS-232	`stream`	`loopback_verified`	any	Read through a pseudo-terminal in the test suite. No physical port exercised.

Interface	Driver	Status	Platforms	Notes
RS-485	`stream`	`implemented`	any	Requires an adapter that handles half-duplex direction.
Modbus RTU	`modbus`	`implemented`	any	Register decoding is unit tested. Bus transaction not exercised.
Modbus TCP	`modbus`	`loopback_verified`	any	Read against a software Modbus TCP server in the test suite. Register decoding unit tested.
CAN	`can`	`loopback_verified`	any	Decoded from a python-can virtual bus. Listen-only by default where the interface supports it.
CAN FD	`can`	`loopback_verified`	any	Decoded from a python-can virtual bus. Listen-only by default where the interface supports it.
Ethernet	`stream`	`loopback_verified`	any	Same code path as the TCP and UDP loopback tests. Modbus TCP and OPC UA have their own rows.
TCP/IP	`stream`	`loopback_verified`	any	Socket loopback in the test suite, including split, malformed and closed connections.
UDP	`stream`	`loopback_verified`	any	Socket loopback in the test suite.
NI-DAQmx	`ni_daqmx`	`implemented`	windows, linux	Needs NI driver software too. Never executed.
NI-VISA	`visa`	`implemented`	any	Uses NI-VISA when present, else PyVISA-py. Never executed.
GPIB	`visa`	`implemented`	any	Through a VISA GPIB resource string. Never executed.
OPC UA	`opcua`	`loopback_verified`	any	Read against a local asyncua server in the test suite.
Vendor adapter	`stream`	`loopback_verified`	any	JSON lines verified over UDP loopback. A vendor SDK bridge in any language can emit these. Alternatively add a driver plugin.
Simulation	`simulator`	`simulated`	any	Engine rig model with fault injection.

4.2 Adding your equipment

Keep this up to date as the lab changes, so that a sensor or DAQ swap is a matter of updating the record.

1. Press **Add lab equipment** and describe the device.
2. Configure and connect it, and compare its readings against a trusted reference.
3. Select its row, press **Record bench validation**, and enter the date, your name, firmware, driver version, evidence and result.

A passing record is the only way a device reaches `bench_validated`. If its driver later changes major version, the row is flagged for revalidation. **Write matrix document** regenerates `docs/HARDWARE_COMPATIBILITY.md`.

5 Measurement channels

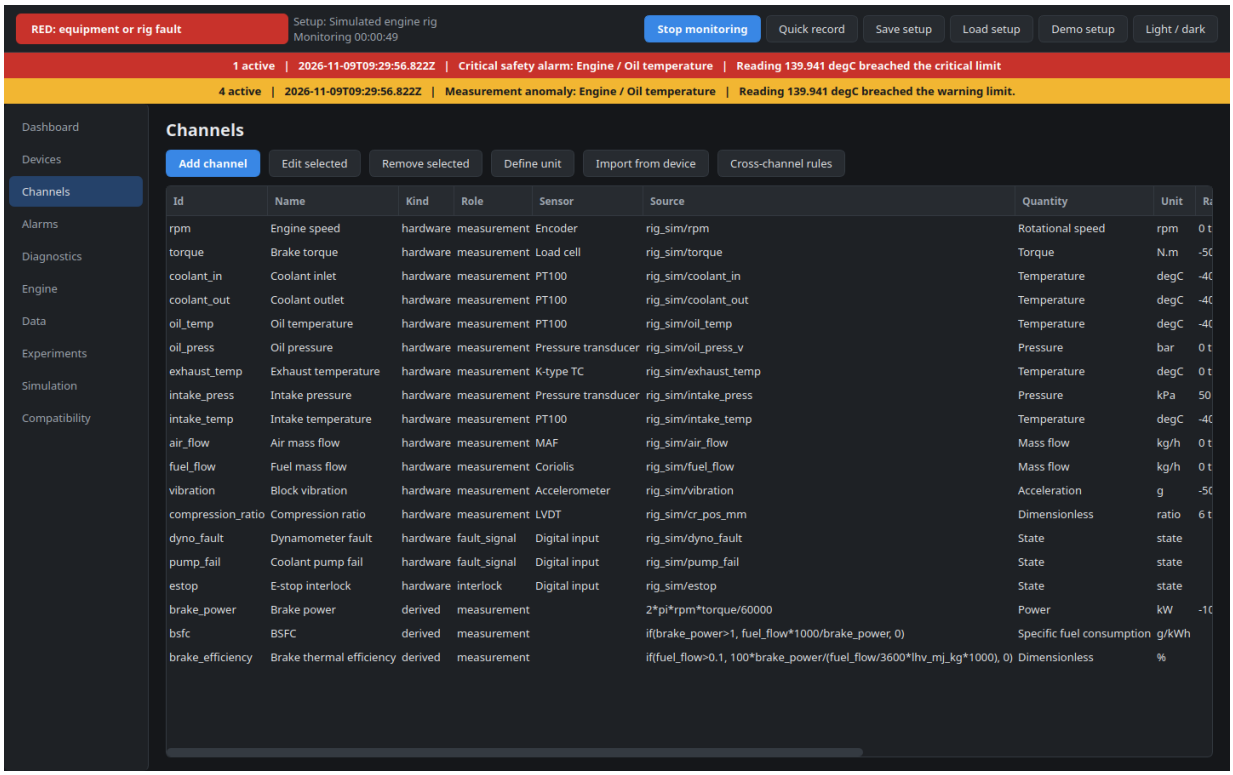


Figure 4: Channels page. Each row is one user-defined measurement.

Nothing in the application is specific to engines. A channel is whatever you define it to be. Monitoring must be stopped to change channels. **Add channel** and **Edit selected** open the channel editor, which has four tabs. **Import from device** creates channels for what a device reports.

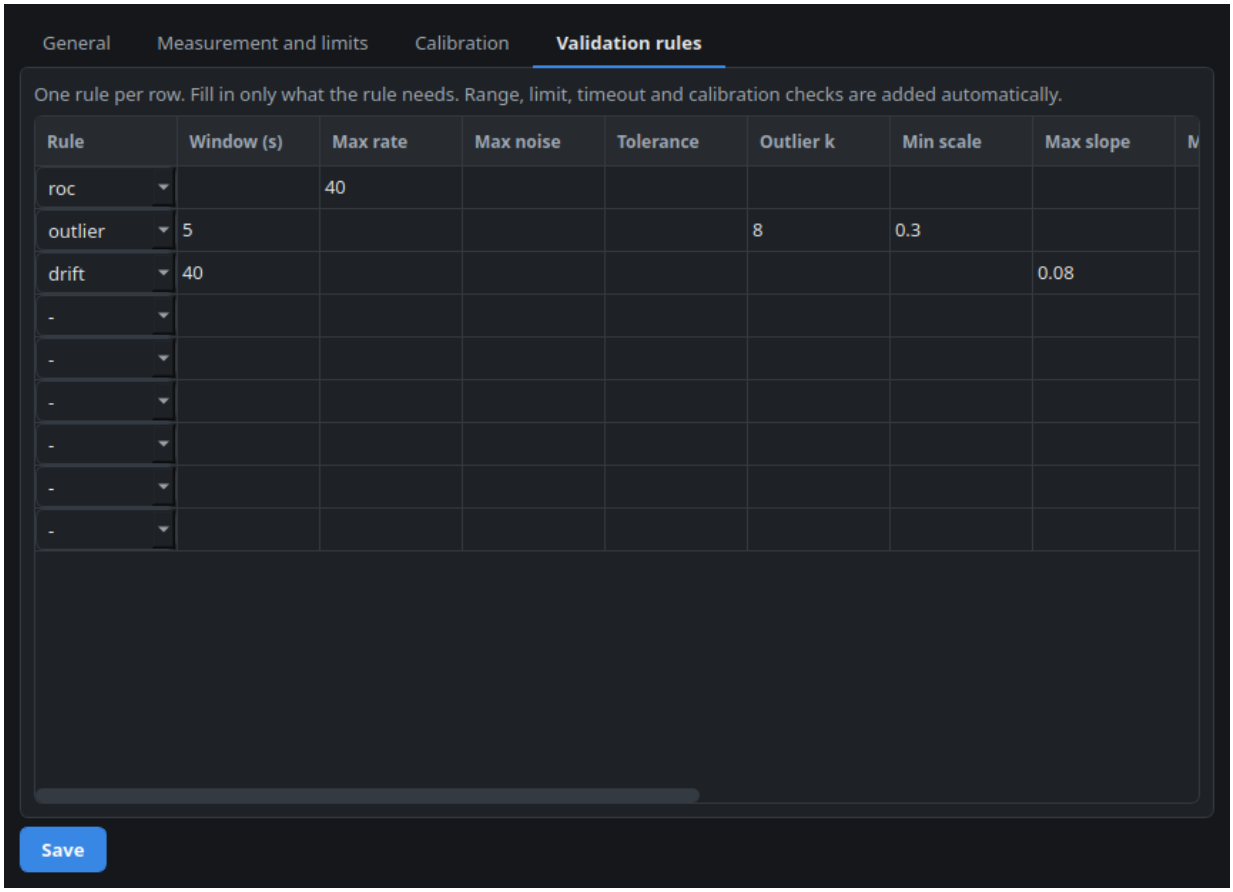


Figure 5: Channel editor. Everything is set through fields, lists and tables.

Field	Meaning
General	
Identifier	Permanent name used in formulas and recorded data. Letters, digits and underscores. Cannot be changed later.
Display name	Change it freely.
Kind	`hardware`, `derived` (from a formula) or `manual` (entered by the operator).
Role	`measurement`, `fault_signal` (equipment reports a fault) or `interlock`.
Sensor type, Equipment	Descriptive. Alarms use the equipment name to collect supporting measurements.
Device, Hardware channel	Where the data comes from.
Formula	For derived channels.
Critical	Tick if losing this channel should be a red condition.
Measurement and limits	
Quantity, Unit, Raw signal unit	Physical quantity and units.
Sample rate	Nominal rate. Used to check sampling intervals.
Sensor range	The sensor's measurement range. Outside it, readings are flagged.
Accuracy, Resolution	When known.
Warning low and high	Yellow limits.
Critical low and high	Red limits.

Field	Meaning
Asserted level	Level treated as active, for fault signals and interlocks.

Leave a field blank if it does not apply. To add a unit, press **Define unit**; conversion between different quantities is always refused.

5.1 Calibration

Raw values are always kept. Calibration produces the separate engineering value. On the **Calibration** tab, choose a type:

linear Enter gain and offset as *Coefficients*, for example `2.5, -1.25`.
polynomial Enter coefficients with the constant term first.
table Fill in raw and value pairs. A table is never extrapolated.

Record the certificate, dates and who performed it. An overdue calibration, or a raw value outside the calibrated low and high, raises a yellow alarm.

5.2 Derived measurements

Set **Kind** to `derived` and enter a **Formula** using other channels' identifiers and setup parameters:

```
2*pi*rpm*torque/60000
if(brake_power>1, fuel_flow*1000/brake_power, 0)
```

Operators `+ - * / % ^ < <= > >= == != && || !`
Functions `sin cos tan asin acos atan atan2 sqrt abs exp log log10 floor ceil round sign min max pow if clamp`
Constants `pi e`

A derived value is recalculated at loop rate from the newest input values, without interpolation. It is flagged if any input was flagged. Circular definitions are refused.

6 Alarms

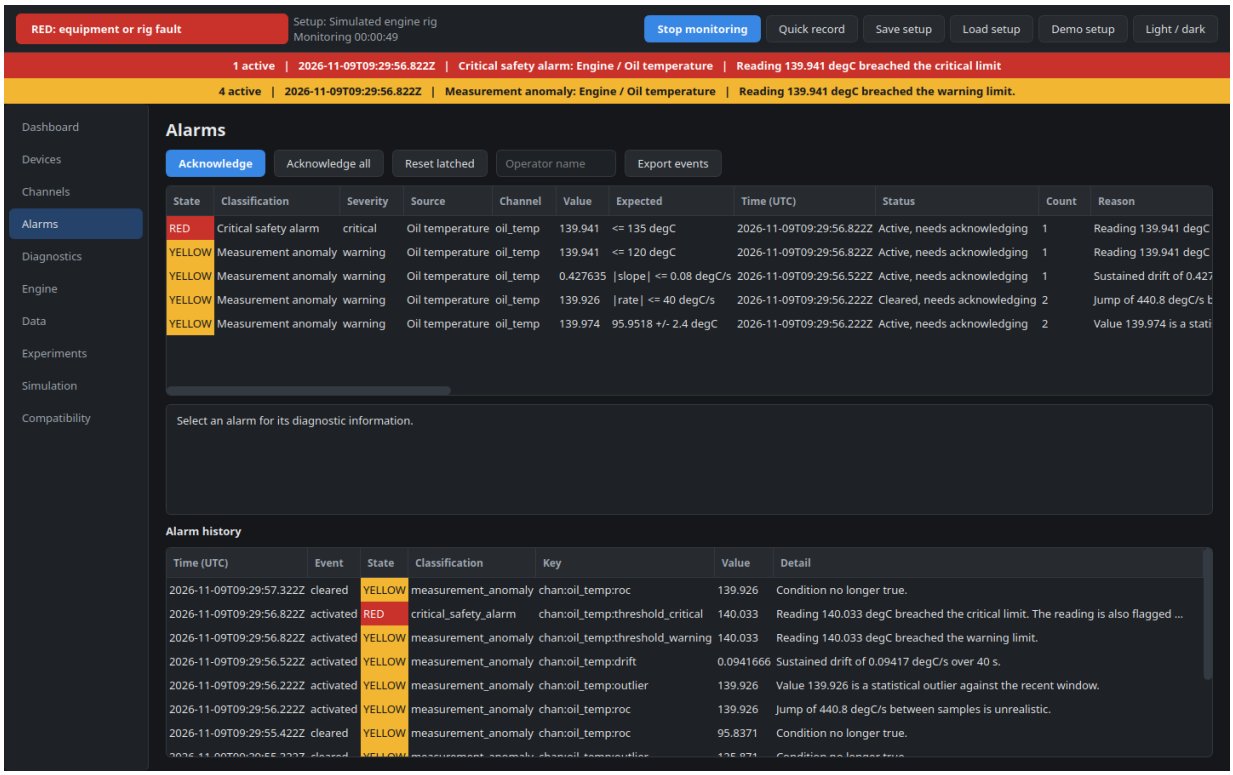


Figure 6: Alarms page with active conditions, details and history.

6.1 The five classifications

Classification	Colour	Used for
Measurement anomaly	Yellow	Suspicious data. Always yellow.
Communication fault	Yellow or red	Link or packet problems. Red only for loss of a critical channel.
Instrument fault	Yellow or red	What the instrument reports about itself. Red for a failure or overload.
Confirmed equipment fault	Red	A dedicated fault signal from the equipment.
Critical safety alarm	Red	A critical limit or an interlock.

A suspicious reading is never treated as a mechanical failure. The application refuses any rule that claims a confirmed equipment fault unless a dedicated fault signal is part of it.

If a critical limit is breached by a reading that is also flagged as suspect, the alarm stays red and says so. The software cannot prove the sensor is wrong, so that decision is left to you.

6.2 Alarm lifecycle

1. A condition must hold for its *persistence* period to become an alarm. Limits default to 0.5 s, so one bad sample does not reach red. Anything shorter is still recorded in the history as a *transient*.
2. An active alarm stays visible until acknowledged. **Acknowledge** confirms that you have seen it; it does not clear it.
3. Once the condition has been false for its *clear delay*, the alarm counts as cleared.

4. Red alarms latch: after they are acknowledged and the condition has cleared, press **Reset latched**. Reset is refused while the condition is still true.

Alarms cannot be shelved, disabled or suppressed. Each event is written to `workspace/logs/`, and to the run when recording. **Export events** saves the history as CSV.

6.3 Reading an alarm

Select a row to see the reason, expected range, recommended action, affected equipment and the supporting measurements captured when the alarm activated.

7 Validation rules

Rules are set per channel. These are automatic:

Rule	When it applies
`range`	When a measurement range is set. Also catches non-numeric readings.
`threshold`	When warning or critical limits are set.
`timestamp`	Hardware channels. Time running backwards or in the future.
`missing`	Hardware channels. Timeout is the larger of 1 s and 10 sample periods.
`missing` (critical)	Critical channels. Red after the larger of 2 s and 20 sample periods.
`calibrationCheck`	When a calibration is set.
`faultSignal`	Channels with the `fault_signal` or `interlock` role.

Add these on the **Validation rules** tab of the channel editor, one row per rule, filling in only the columns that rule uses:

Type	Parameters	Detects
`roc`	`maxRate`	Unrealistic jumps between samples
`stuck`	`windowS`, `tolerance`	Frozen readings. Use only on signals that always move.
`noise`	`windowS`, `maxStd`	Excessive noise, ignoring slow trends
`outlier`	`windowS`, `k`, `minScale`	Statistical outliers (Hampel test)
`drift`	`windowS`, `maxSlope`	Sustained slope. A real warm-up looks identical, so use it on signals expected to hold steady.
`interval`	`windowS`, `tolerance`, `maxFraction`	Abnormal sampling intervals

Any rule can also set a persistence, clear delay, severity and recommendation. Flagged samples remain in the data with their flags; nothing is removed.

7.1 Cross-channel rules

Channels > Cross-channel rules defines checks that span channels, one row per rule. A rule fires while its condition is true, for example `rpm < 50 && torque > 20` for torque reported while the engine is not turning. Use channel identifiers in the condition.

8 Diagnostics

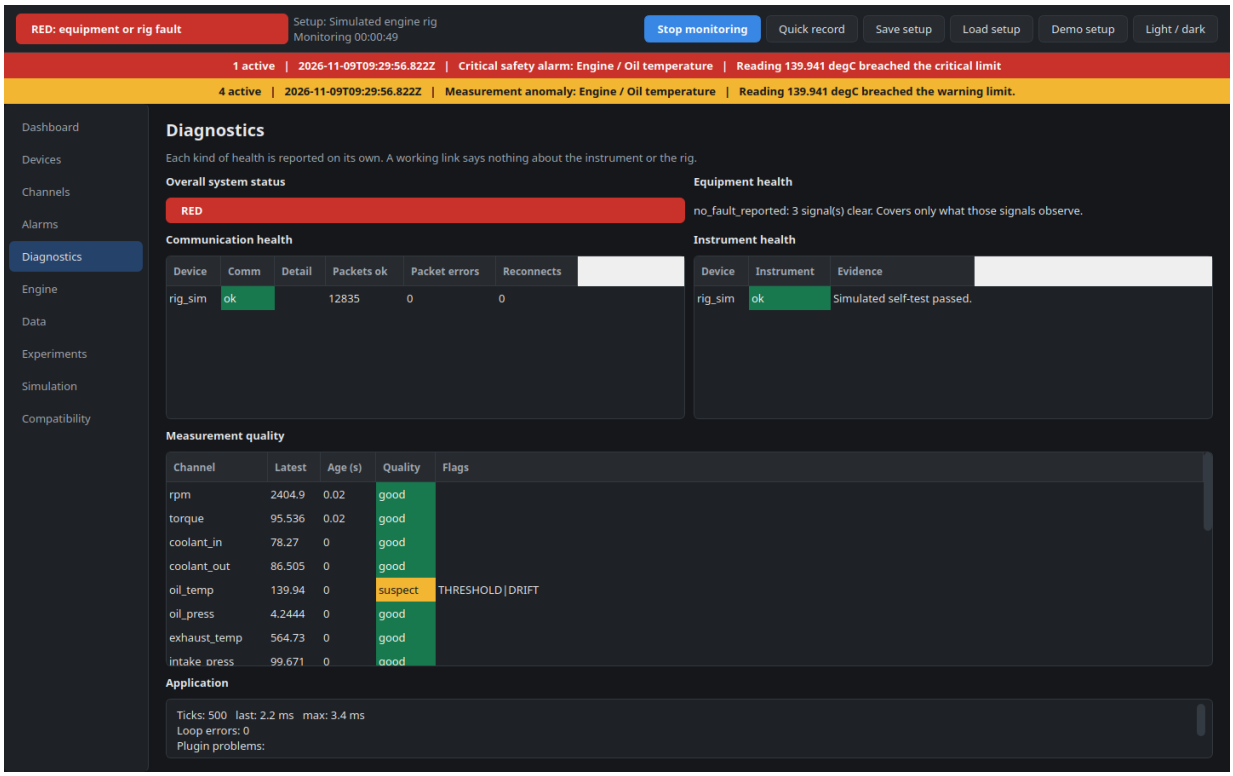


Figure 7: Diagnostics page: each kind of health is reported on its own.

A working link says nothing about whether the instrument or the rig is healthy, so each layer is reported independently.

Layer	Reading it
Communication	`ok`, `degraded`, `lost` or `offline`, with packet and reconnection counts.
Instrument	`ok` only when the device gives real evidence. `not_assessed` means it provides none.
Measurement quality	`good`, `suspect` or `no_data` per channel, with current flags.
Equipment	`no_fault_reported` covers only what the wired signals observe. `not_monitored` means no fault signals are configured.
Application	Loop timing and errors, and plugins that failed to load.

9 Dashboards

Dashboards are collections of pages, each laid out on a 12 by 8 grid.

- **Add widget**: numeric, gauge, trend, XY, table, alarms or status lamps.
- **Edit layout**: drag a block to move it, drag its corner handle to resize, and press **Apply**.
- **Add page** and **Remove page**.
- Enter a name and press **Save**. Pick a saved dashboard from the list at the top left.

Keep one dashboard per engine or experiment. Trends mark flagged samples with a red cross. Dashboards only display data, so changing one cannot affect acquisition.

10 Engine and compression ratio

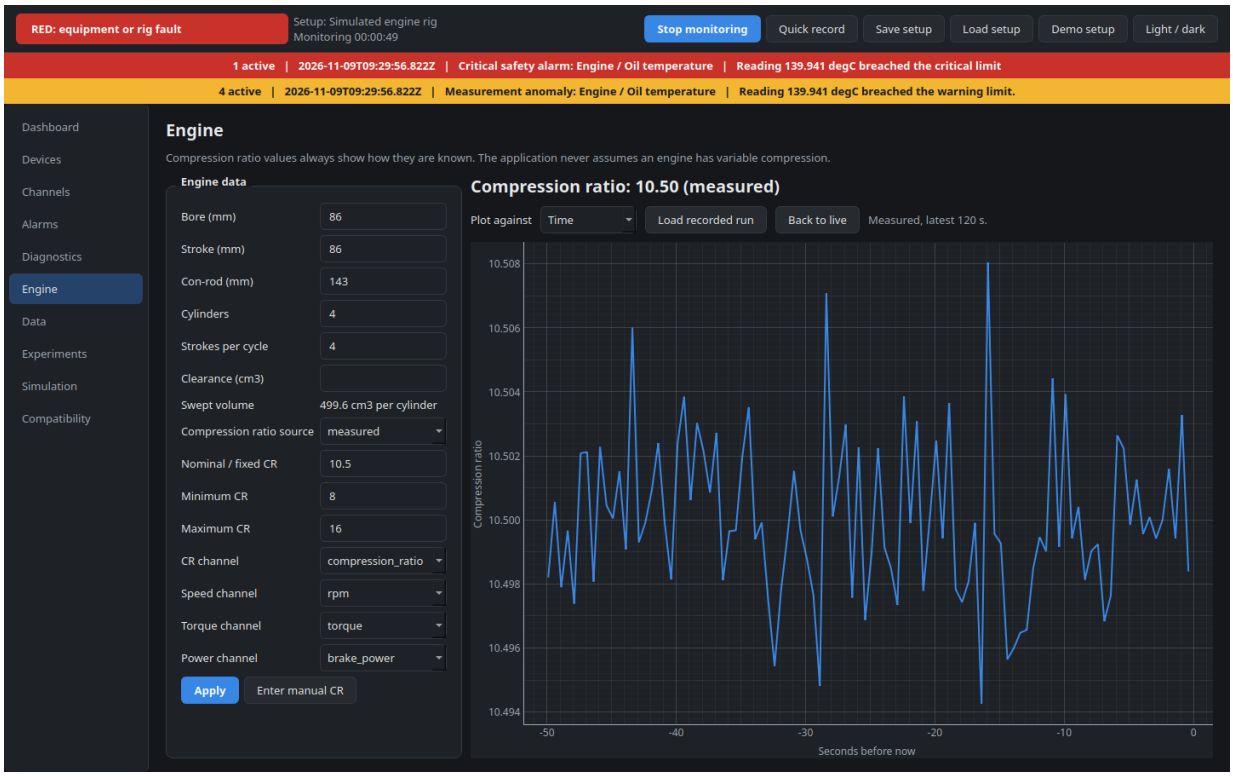


Figure 8: Engine page with a measured compression ratio.

Enter the geometry you know and leave the rest blank. Press **Apply** after changing anything.

10.1 Compression ratio source

You choose the source explicitly. The application never assumes an engine has variable compression.

Source	Use when
`fixed`	The engine has one ratio. Enter it as *Nominal*. Shown as *configured*.
`measured`	A channel reports the actual ratio. Shown as *measured*.
`manual`	The operator adjusts it by hand. Press **Enter manual CR** at each change; entries are stored with the run. Shown as *manual*.
`calculated`	Derived from bore, stroke and clearance volume. Shown as *calculated*.
`unknown`	None of the above.

Plots against time, speed, torque and brake power are available for measured channels. Press **Load recorded run** to plot a saved run.

10.2 Calculations

Standard engine quantities are best set up as derived channels, as in the demo setup, so they are recorded with the run.

$$V_s = \tfrac{\pi}{4} B^2 S \qquad CR = \frac{V_s + V_c}{V_c}$$

$$P\,[\mathrm{kW}] = \frac{2\pi N T}{60000} \qquad \mathrm{BMEP} = \frac{2\pi\, n_r T}{V_d}$$

$$\mathrm{BSFC}\,[\mathrm{g/kWh}] = \frac{1000\,\dot{m}_f}{P} \qquad \eta_b = \frac{P}{\dot{m}_f\,\mathrm{LHV}}$$

where n_r is crank revolutions per cycle. For analysis across a recorded run, and comparison between compression ratios, use Python directly, from the private environment:

```
from edaq.data.store import RunReader
from edaq.rig import Geometry, analyze_run
r = RunReader("workspace/runs/<run folder>")
g = Geometry.from_dict({"boreMm": 86, "strokeMm": 86, "cylinders": 4})
table = analyze_run(r, {"rpm": "rpm", "torque": "torque", "fuel": "fuel_flow"}, g, {"
    lhvMJkg": 42.5})
```

11 Recording and data exchange

Quick record captures to `workspace/runs/`. Each run stores a full copy of the configuration and calibration in force, plus the raw value, converted value and quality flags of every sample. A run interrupted by a crash remains readable.

11.1 Exporting

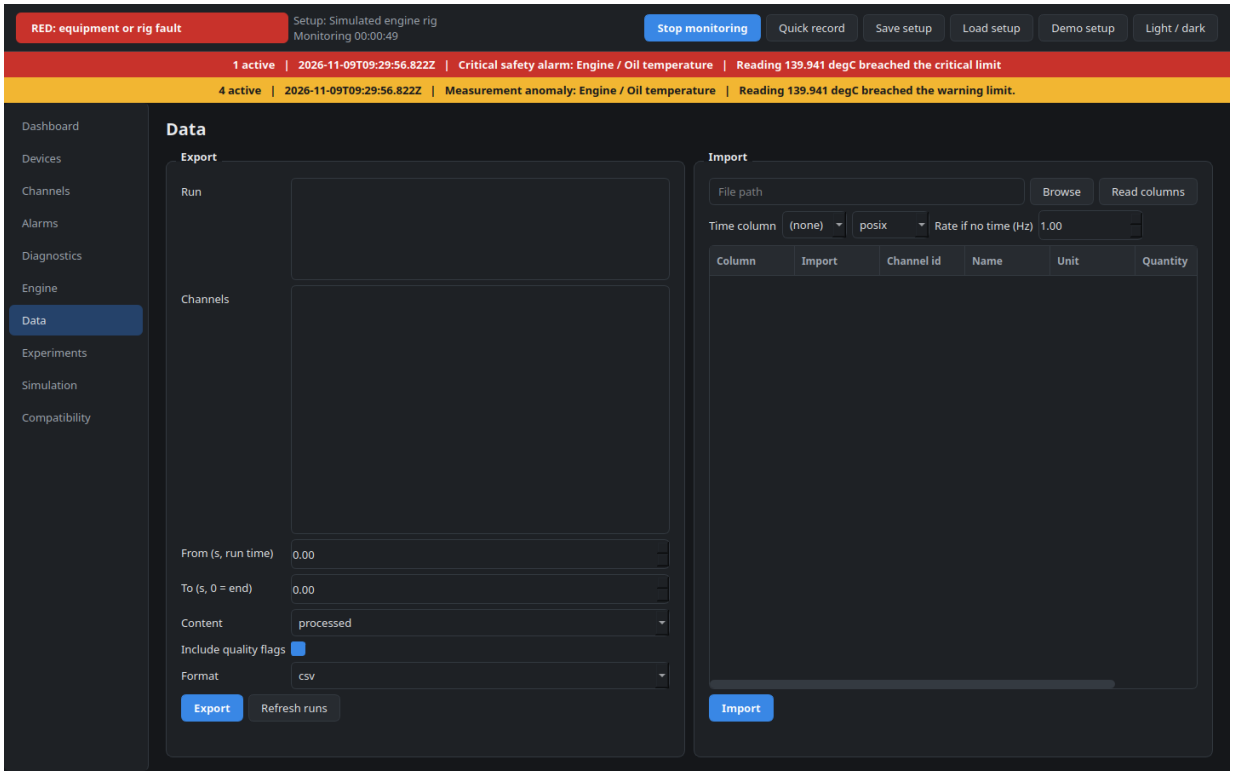


Figure 9: Data page.

Press **Refresh runs**, select a run and channels, set an optional time range, then choose content and format. Files are written to `workspace/exports/`.

- Data: CSV, TXT, XLSX, XLS, JSON, XML, MAT, HDF5, Parquet
- Figures: PDF, PNG, SVG, JPEG

Channels at different rates share a table with blanks where a channel has no sample at that instant. Nothing is resampled. XLSX and XLS have row limits; if a run exceeds them, the export is refused and you can choose a shorter range or another format.

11.2 Importing

Enter a path or press **Browse**, then press **Read columns**. Choose the time column and its kind, or give a sample rate if there is none. Edit the mapping table and press **Import**. The source file's name, hash and original columns stay with the imported run.

12 Experiments and reports

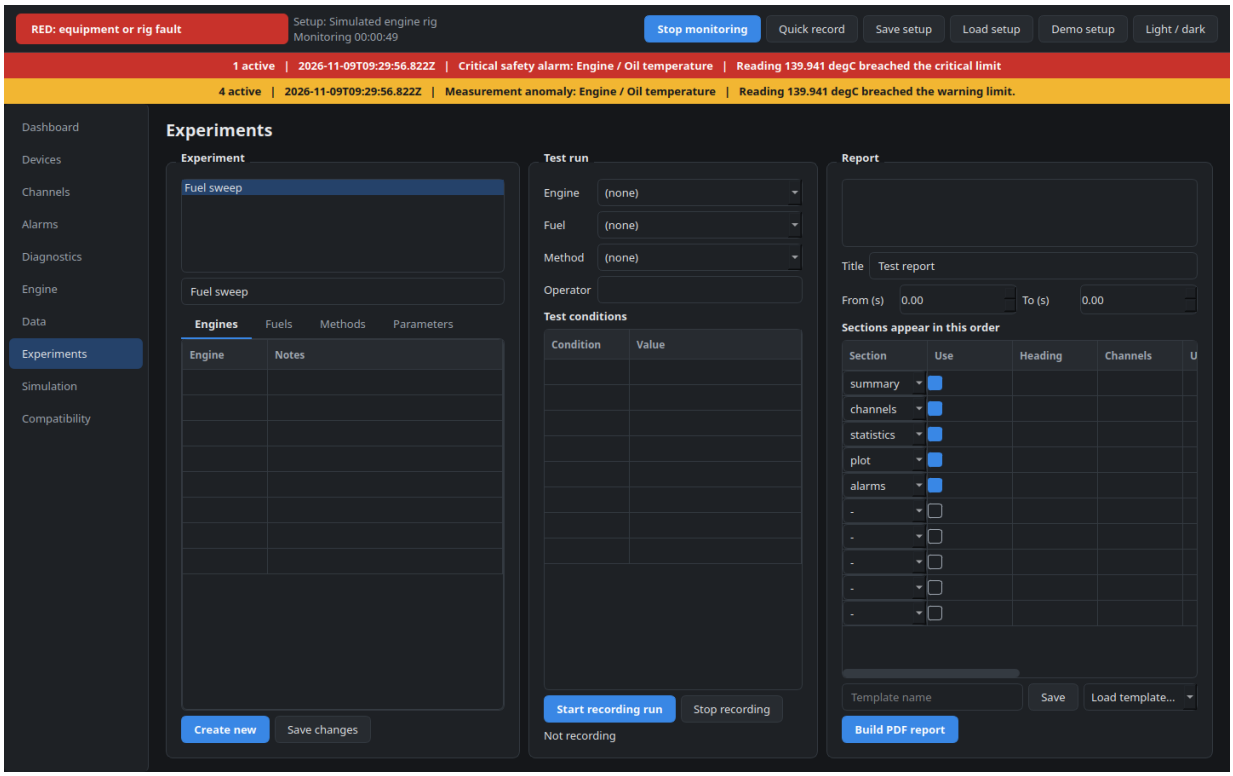


Figure 10: Experiments page.

An experiment describes what is being tested, independently of the hardware.

1. Enter a name and fill in the **Engines**, **Fuels**, **Methods** and **Parameters** tables, then press **Create new**. To change an existing experiment, select it, edit it and press **Save changes**.
2. Start monitoring. Under **Test run**, choose the engine, fuel and method, enter the operator and any test conditions, and press **Start recording run**.
3. Press **Stop recording** at the end of the run. Repeat for further runs.

Later edits to an experiment do not change runs already recorded.

12.1 Reports

Select a run under **Report**. Each row of the sections table is one section of the report, in that order. Change a row's **Section** to reorder them, and clear **Use** to leave one out. Then press **Build PDF report**.

Section	Columns used	Content
`summary`		Identification and provenance
`channels`	Channels	Channel and calibration table
`statistics`	Channels, Units	Mean, min, max, standard deviation, counts
`alarms`		Alarm events
`plot`	Channels, Heading	Time plots
`text`	Heading, Text	Free text

List channels by identifier, separated by commas. Blank means all channels. Write display units as, for example, `oil_press=psi`. **From** and **To** limit the report to part of the run. Enter a name and press **Save** to keep a report layout, and choose it from the **Load** list to reuse it.

13 Simulation

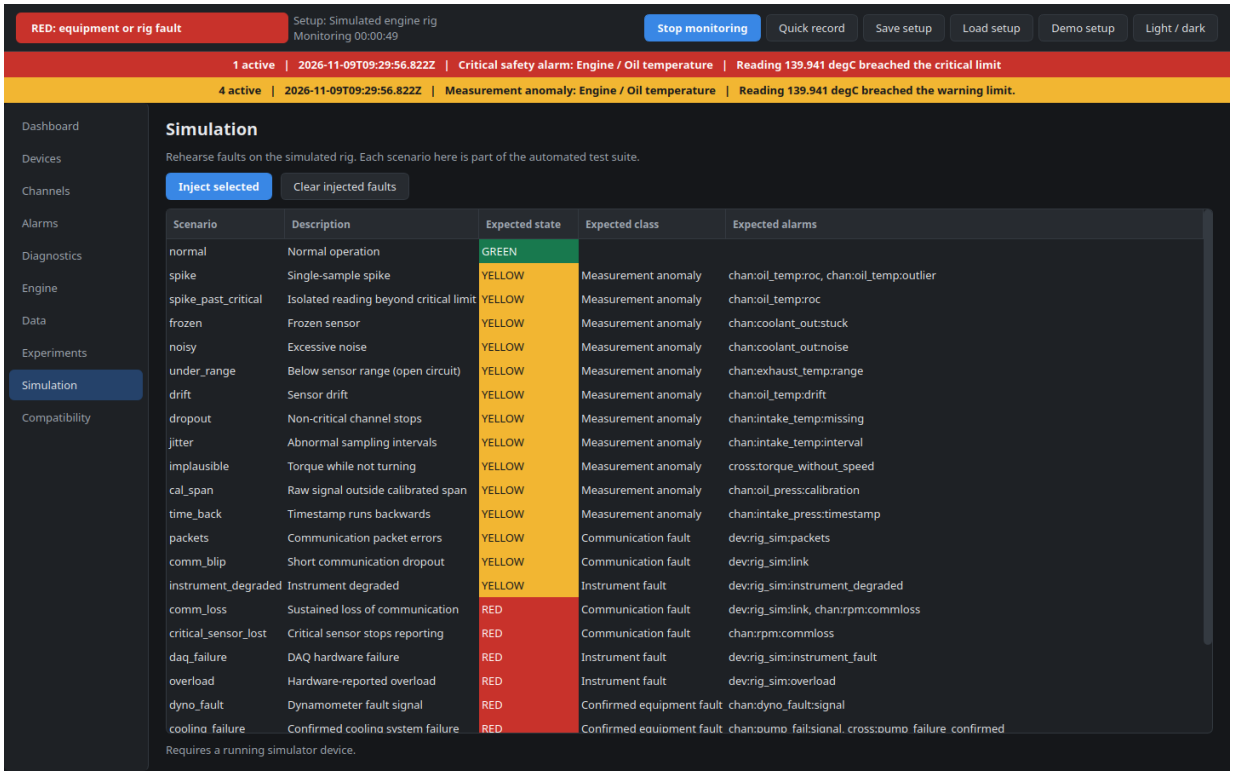


Figure 11: Simulation page.

With the simulated device running, select a scenario and press **Inject selected**. Use this to train operators and to check your thresholds. **Clear injected faults** removes faults, though latched alarms still need acknowledging. Each of these scenarios is part of the automated test suite, against the state and classification shown here.

Scenario	Id	State	Classification
Normal operation	`normal`	GREEN	None

Scenario	Id	State	Classification
Single-sample spike	`spike`	YELLOW	Measurement anomaly
Isolated reading beyond critical limit	`spike_past_critical`	YELLOW	Measurement anomaly
Frozen sensor	`frozen`	YELLOW	Measurement anomaly
Excessive noise	`noisy`	YELLOW	Measurement anomaly
Below sensor range (open circuit)	`under_range`	YELLOW	Measurement anomaly
Sensor drift	`drift`	YELLOW	Measurement anomaly
Non-critical channel stops	`dropout`	YELLOW	Measurement anomaly
Abnormal sampling intervals	`jitter`	YELLOW	Measurement anomaly
Torque while not turning	`implausible`	YELLOW	Measurement anomaly
Raw signal outside calibrated span	`cal_span`	YELLOW	Measurement anomaly
Timestamp runs backwards	`time_back`	YELLOW	Measurement anomaly
Communication packet errors	`packets`	YELLOW	Communication fault
Short communication dropout	`comm_blip`	YELLOW	Communication fault
Instrument degraded	`instrument_degraded`	YELLOW	Instrument fault
Sustained loss of communication	`comm_loss`	RED	Communication fault
Critical sensor stops reporting	`critical_sensor_lost`	RED	Communication fault
DAQ hardware failure	`daq_failure`	RED	Instrument fault
Hardware-reported overload	`overload`	RED	Instrument fault
Dynamometer fault signal	`dyno_fault`	RED	Confirmed equipment fault
Confirmed cooling system failure	`cooling_failure`	RED	Confirmed equipment fault
Critical temperature exceeded	`over_temp`	RED	Critical safety alarm
Critical pressure limit	`low_oil_press`	RED	Critical safety alarm
Equipment interlock activated	`interlock`	RED	Critical safety alarm

14 Saving setups

A setup holds devices, channels, rules, parameters and engine data. You can keep as many as you need, for example one for each engine, rig or experiment.

Save setup and **Load setup** both open the setup library, which lists every saved setup.

Save Enter a name and press **Save**. To replace an existing setup, press **Save** a second time when asked.
Load With monitoring stopped, select a setup and press **Load**. If it cannot be loaded, the current setup stays in place and the reason is shown.
Delete Select a setup and press **Delete** twice. This cannot be undone.
Open file Loads a setup file from anywhere, for example one copied from another computer. Setups saved by the MATLAB build can be opened too.

Names may use letters, digits and spaces, and must start with a letter. Spaces are stored as underscores. **Demo setup** returns to the simulated rig.

Whatever you last worked on reopens at the next launch, whether or not you saved it under a name. Saved setups are not changed until you save over them.

If the saved setup cannot be used, for example because it needs a driver that is not available on this computer, the application still starts. It loads the demo setup, tells you why, and keeps your file in `workspace/setups` under a name beginning `rejected_`. To use that file, make the driver available or edit the file, then load it with **Load setup > Open file**.

15 Troubleshooting

Symptom	What to check
The first launch stops with an error	It needs internet access. On Debian or Ubuntu the `python3-venv` package may be missing. Fix that and start it again.
A driver shows as unavailable	The reason is on the **Compatibility** page: a library that could not be installed, or the wrong operating system.
A warning at launch and the demo setup appears	The saved setup could not be used on this computer. The warning gives the reason, and your file is kept (Section 14).
A device keeps reconnecting	**Devices**, *Last error*. Check cable, power, address and baud rate, then run **Test device** with monitoring stopped.
Connected, but no data	Frame format, terminator and channel table. A device with a poll command needs the right command.
Many packet errors	Delimiter or format mismatch; otherwise cabling, termination or bus load.
All channels of one device report missing	The device link: those alarms are classed as communication faults.
A `stuck` alarm on a steady signal	Raise its tolerance or lengthen its window, or remove the rule from that channel.
A `drift` alarm during warm-up	Expected. Raise its maximum slope or leave the rule off that channel.
The display is slow	See the tick times on **Diagnostics**. Reduce widgets and shorten trend windows.
"Stop monitoring before changing the configuration"	Press **Stop monitoring** first.

16 Known limitations

- All devices share one acquisition loop running at up to 20 Hz, reading buffered data. A slow device delays the others.
- The application is not designed for crank-angle-resolved combustion measurement at high sample rates.
- Text-stream, Modbus, VISA and OPC UA timestamps are the time of arrival at the computer, not at the sensor.
- The built-in ISO 1585 power correction covers naturally aspirated spark-ignition engines only.

17 Where your files are

Settings and run details are stored as Python code files you can open and read. Change them through the application; it loads only plain values from these files, so a file containing anything else is refused.

Folder	Contents
`workspace/setups`	One file per saved setup. `current.py` is the last working state.
`workspace/dashboards`	Dashboards
`workspace/device_profiles`	Device profiles
`workspace/runs`	Quick recordings and imports
`workspace/experiments`	Experiments and their runs
`workspace/exports`	Exported files and reports
`workspace/logs`	Alarm event logs

Folder	Contents
`config/hardware_compatibility.py`	Compatibility matrix
`plugins/drivers`	Hardware drivers

18 Licence

Copyright © 2026 Kabir Dhingra.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, version 3. The full text is in the `LICENSE` file.

This program is distributed in the hope that it will be useful, but **without any warranty**; without even the implied warranty of merchantability or fitness for a particular purpose.